

CL-2005: Database Systems

Lab # 14: Introduction To Mongo DB

Objective:

To learn Mongo DB.

Scope:

The student shall know the following:

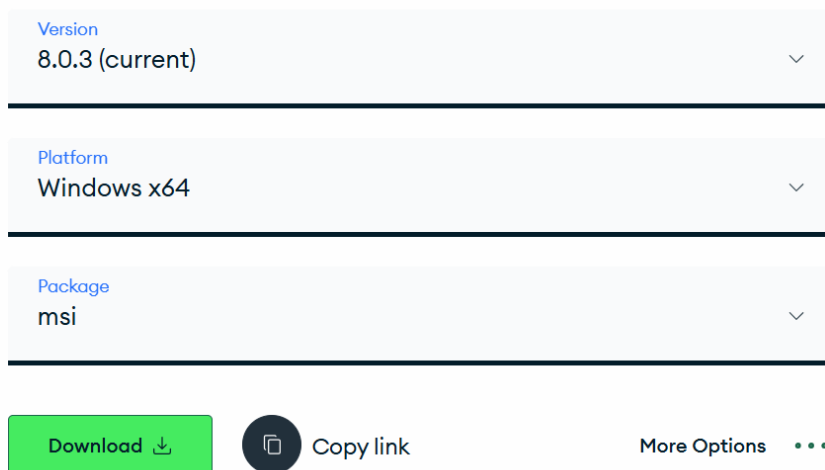
- SQL Commands.
- SQL Queries
- SQL schema
- Hands-on experience with the above-mentioned concepts.

Installation:

Click on the below link to download Mongo DB community server.

<https://www.mongodb.com/try/download/community>

Now from below screen shot, select following requirements as shown below.

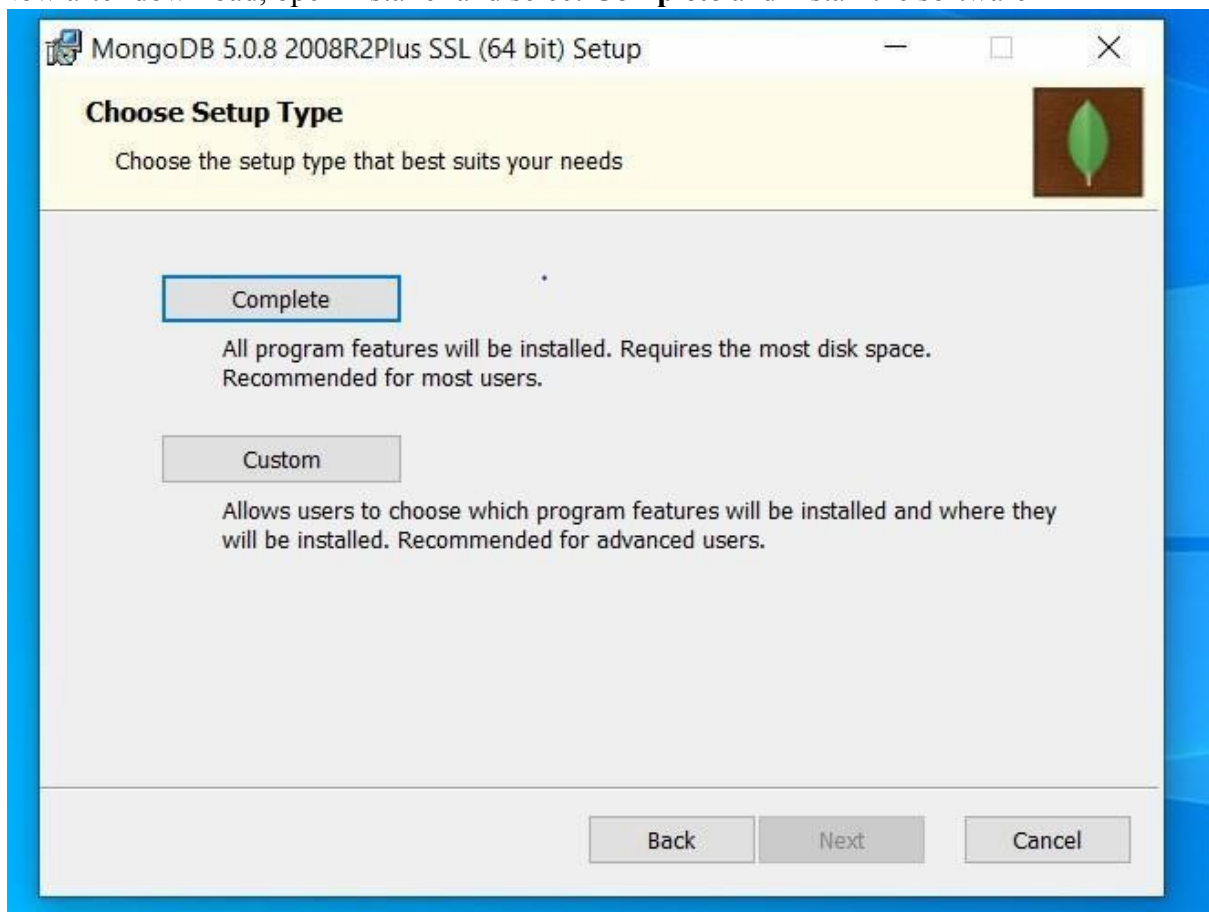


The screenshot shows the MongoDB download interface. It features three dropdown menus for selecting the version, platform, and package. The version is set to 8.0.3 (current), the platform is Windows x64, and the package is msi. Below the dropdowns are three buttons: a green 'Download' button with a download icon, a 'Copy link' button with a copy icon, and a 'More Options' button with a three-dot menu icon.

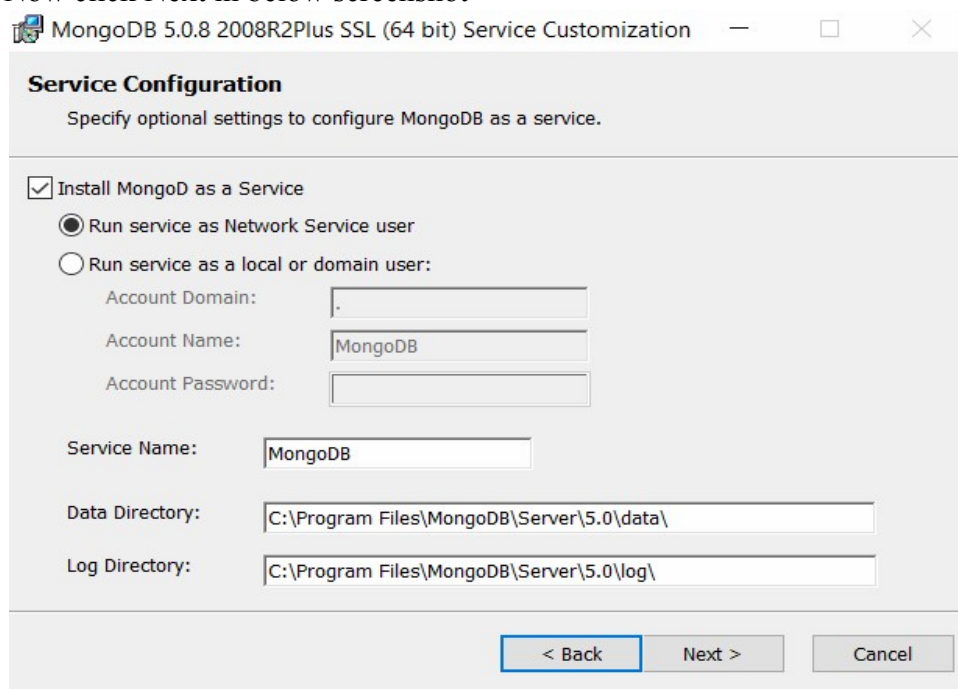
Version	8.0.3 (current)	▼
Platform	Windows x64	▼
Package	msi	▼

[Download](#)  [Copy link](#) [More Options](#) 

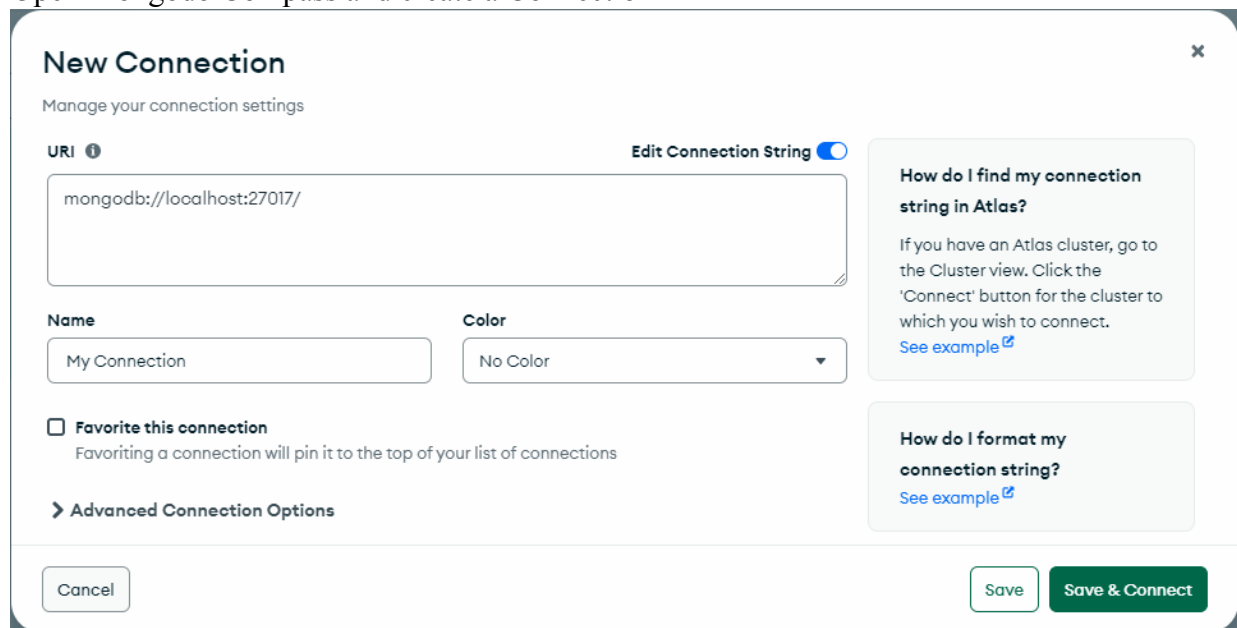
Now after download, open installer and select **Complete** and install the software



Now click Next in below screenshot

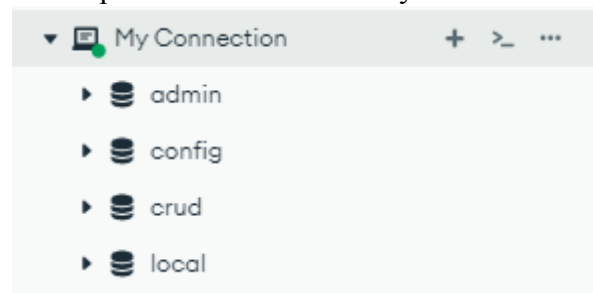


Open MongoDB Compass and create a Connection



The screenshot shows the 'New Connection' dialog in MongoDB Compass. The title is 'New Connection' with a close button (X) in the top right. Below the title is the subtitle 'Manage your connection settings'. The main form has a 'URI' field with a help icon (i) and an 'Edit Connection String' toggle switch. The URI field contains 'mongodb://localhost:27017/'. Below the URI field are two fields: 'Name' with the value 'My Connection' and 'Color' with a dropdown menu showing 'No Color'. There is a checkbox for 'Favorite this connection' with the text 'Favoriting a connection will pin it to the top of your list of connections'. Below this is a link '> Advanced Connection Options'. At the bottom are three buttons: 'Cancel', 'Save', and 'Save & Connect'. On the right side of the dialog, there are two informational boxes. The first box is titled 'How do I find my connection string in Atlas?' and contains text about finding the connection string in the Atlas Cluster view, with a link 'See example'. The second box is titled 'How do I format my connection string?' and contains a link 'See example'.

Your new connection will be shown in your Connections Explorer on the left as such. From there open the terminal to run your command.



You can run **show dbs** to see your storage. As we have no database our result is following:

```
> show dbs
< admin    40.00 KiB
  config  108.00 KiB
  local    80.00 KiB
test> |
```

The USE Command:

MongoDB **use DATABASE_NAME** is used to create database. The command will create a new database, if it doesn't exist otherwise, it will return the existing database Basic syntax of use DATABASE statement is as follows:

```
> use NEW_DB
< switched to db NEW_DB
NEW_DB> |
```

The DROP Command:

MongoDB **db.dropDatabase()** command is used to drop a existing database. Basic syn tax of dropDatabase () command is as follows:

```
> db.dropDatabase()
< { ok: 1, dropped: 'NEW_DB' }
```

Creating Collection in Mongo DB:

MongoDB **db.createCollection(name, options)** is used to create collection. Basic syntax of createCollection() command is as follows

```
NEW_DB> db.createCollection(name,options)
```

In the command, **name** is name of collection to be created. **Options** is a document and used to specify configuration of collection

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

Options parameter is optional, so you need to specify only name of the collection. Following is the list of options you can use:

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a collection fixed size colleccction that automatically overwrites its oldest entries when it reaches its maximum size. If you specify true, you need to specify size parameter also.
autoIndexID	Boolean	(Optional) If true, automatically create index on _id field.s Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. If capped is true, then you need to specify this field also.
max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

Example:

Below we have created collection “Users”

```
> db.createCollection("Users")
< { ok: 1 }
```

We can also see list of all collections in our database by using ‘show collections’.

```
> show collections
< Users
```

Dropping Collection in Mongo DB:

You can drop any collection by the following command:

```
> db.createCollection("temp")
< { ok: 1 }
> db.temp.drop()
< true
```

Mongo DB – Insert Document:

To insert data into MongoDB collection, you need to use MongoDB's insert() or save() method.

Basic syntax of insert() command is as follows:

```
> db.Users.insert({name:"Akbar",age:25,phone:03043165827})
< DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('674cc77aa9691de20f56c090')
  }
}
```

If the collection doesn't exist in the database, then MongoDB will create this collection and then insert document into it.

Mongo DB – Find Method:

To query data from MongoDB collection, you need to use MongoDB's find() method.

Basic syntax of find() method is as follows

```
> db.Users.find()
< {
  _id: ObjectId('674cc77aa9691de20f56c090'),
  name: 'Akbar',
  age: 25,
  phone: 3043165827
}
```

find() method will display all the documents in a non-structured way.

Alternatively, you can use Pretty method to display in structured form.

```
> db.Users.find().pretty()
```

RDBMS Where Clause Equivalents in MongoDB

To query the document on the basis of some condition, you can use following operations

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>:<value>}	db.mycol.find({"by":"tutorials point"},pretty())	where by = 'tutorials point'
Less Than	{<key>:{\$lt:<value>}}	db.mycol.find({"likes":{\$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{\$lte:<value>}}	db.mycol.find({"likes":{\$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{\$gt:<value>}}	db.mycol.find({"likes":{\$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{\$gte:<value>}}	db.mycol.find({"likes":{\$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{\$ne:<value>}}	db.mycol.find({"likes":{\$ne:50}}).pretty()	where likes != 50

Mongo DB – AND statement:

In the find() method if you pass multiple keys by separating them by ',' then MongoDB treats it AND condition. Basic syntax of AND is shown below:

```
NEW_DB> db.Users.find({key1:value1,key2:value2})
```

Mongo DB – OR statement:

To query documents based on the OR condition, you need to use \$or keyword. Basic syntax of OR is shown below:

```
NEW_DB> db.Users.find({$or:[{key1:value1},{key2:value2}]})
```

Example:

```
> db.Users.find({$or:[{age:21},{age:25}]});  
< {  
  _id: ObjectId('674cc77aa9691de20f56c090'),  
  name: 'Akbar',  
  age: 25,  
  phone: 3043165827  
}
```

Mongo DB – AND - OR Together:

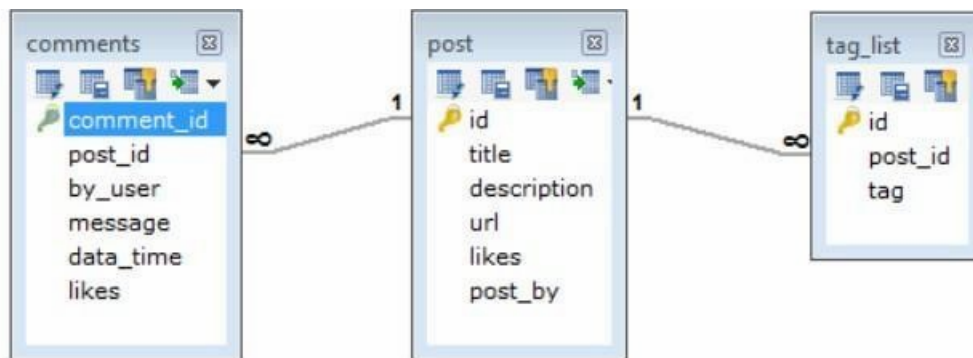
Below given example will show the documents that have age greater than 18 and whose name is either 'Akbar' or 'Bilal'.

```
> db.Users.find({age:{$gt:18},$or:[{name:"Akbar"},{name:"Bilal"}]})  
< {  
  _id: ObjectId('674cc77aa9691de20f56c090'),  
  name: 'Akbar',  
  age: 25,  
  phone: 3043165827  
}
```

EXAMPLE SCHEMA IN MONGO DB

Example Suppose a client needs a database design for his blog website and see the differences between RDBMS and MongoDB schema design. Website has the following requirements.

- Every post has the unique title, description and url.
- Every post can have one or more tags.
- Every post has the name of its publisher and total number of likes.
- Every Post have comments given by users along with their name, message, data-time and likes.
- On each post there can be zero or more comments. In RDBMS schema design for above requirements will have minimum three tables.



While in MongoDB schema design will have one collection post and has the following structure:

```

{
  _id: POST_ID
  title: TITLE_OF_POST,
  description: POST_DESCRIPTION,
  by: POST_BY,
  url: URL_OF_POST,
  tags: [TAG1, TAG2, TAG3],
  likes: TOTAL_LIKES,
  comments: [
    {
      user:'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    },
    {
      user:'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    }
  ]
}
  
```


TASK

- You have to create database which has following tables

Employee Table (Creation):

```
CREATE TABLE EMP
(EMPNO NUMERIC(4) NOT NULL,
ENAME VARCHAR(10),
JOB VARCHAR(9),
MGR NUMERIC(4),
HIREDATE DATE,
SAL NUMERIC(7, 2),
COMM NUMERIC(7, 2),
DEPTNO NUMERIC(2));
```

Department Table (Creation):

```
CREATE TABLE DEPT
(DEPTNO NUMERIC(2),
DNAME VARCHAR(14),
LOC VARCHAR(13));
```

- Now convert above RDBMS into MongoDB schema and create collection.
- Insert at least 6 documents.

Display following queries

- List all the employee names, their salaries.
- List all the employee names, jobs, and salaries who have not been given any commission.
- List all the employee identity numbers, names, jobs, and salaries are greater than 1500
- List all the employee names and jobs who are not CLERK, ANALYST, and SALESMAN.